jQuery Notes (Updated – Complete)

1. Introduction

jQuery is a JavaScript library that simplifies DOM manipulation, event handling, data access, and AJAX.

Key Idea: Write less code to perform common JavaScript tasks.

2. Setup

Method 1: CDN (Recommended)

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

Method 2: Local File

1. Download jQuery from official site
2. Include in project:

```
<script src="jquery.min.js"></script>
```

3. Basic Syntax

```
$(selector).action();
```

- `$` → jQuery object
- `selector` → selects HTML element
- `action()` → operation

4. Document Ready

```
$(document).ready(function() {
    // code here
});
```

Ensures DOM is fully loaded before execution.

5. Selectors

```
$("#id")        // ID selector
$(".class")     // Class selector
$("p")          // Element selector
```

---

6. Reading Data (IMPORTANT)

Read Input Value

```
<input type="text" id="name">
```

```
let value = $("#name").val();
console.log(value);
```

Read Text Content

```
<p id="text">Hello</p>
```

```
let text = $("#text").text();
console.log(text);
```

Read HTML Content

```
let html = $("#text").html();
```

Read Attribute

```
<img id="img" src="image.png">
```

```
let src = $("#img").attr("src");
```

---

7. Writing / Updating Data

Set Input Value

```
$("#name").val("John");
```

Set Text

```
$("#text").text("New Text");
```

Set HTML

```
$("#text").html("<b>Bold Text</b>");
```

Set Attribute

```
$("#img").attr("src", "new.png");
```

Set CSS

```
$("#box").css("color", "red");
```

---

8. Events

```
$("#btn").click(function() {
    alert("Clicked");
```

```
});
```

---

9. Show / Hide

```
$("#box").hide();
$("#box").show();
$("#box").toggle();
```

---

10. Effects

```
$("#box").fadeOut();
$("#box").fadeIn();

$("#box").slideUp();
$("#box").slideDown();
```

---

11. Method Chaining

```
$("#box")
    .text("Hello")
    .css("color", "red")
    .slideUp()
    .slideDown();
```

---

12. Example: Read + Write (Form Demo)

HTML

```
<input type="text" id="name" placeholder="Enter name">
<button id="btn">Submit</button>
<p id="output"></p>
```

---

jQuery

```
$("#btn").click(function() {

    // Read value
    let name = $("#name").val();

    // Write value
    $("#output").text("Hello " + name);

});
```

---

13. Example: Attribute Read + Update

```
<img id="img" src="old.png">
<button id="change">Change Image</button>
```

```
$("#change").click(function() {

    let current = $("#img").attr("src"); // read

    console.log(current);

    $("#img").attr("src", "new.png"); // update

});
```

AJAX Notes (Complete)

---

1. Introduction

AJAX

Full Form: Asynchronous **J**avaScript **A**nd **X**ML

---

2. What is AJAX?

AJAX is a technique used to send and receive data from a server **without reloading the web page**.

Key Idea

- Communicates with server in the background
- Updates only required parts of the page
- Improves performance and user experience

---

3. Traditional vs AJAX Approach

Without AJAX (Form Submission)

- Request sent to server
- Entire page reloads
- New page is loaded with response

With AJAX

- Request sent in background
- No page reload
- Only specific part of page updates

---

4. What is an API?

API (Application Programming Interface) is a system that allows communication between **frontend and backend**.

Simple Definition

> API is a bridge between client (browser) and server (database/backend)

---

5. How AJAX Works with API

1. User performs action (click, input)
2. JavaScript sends request (AJAX)
3. API receives request
4. Server processes data
5. API sends response (usually JSON)
6. UI updates dynamically

---

6. HTTP Methods (API Methods)

These define the type of operation performed on the server.

Method	Purpose	Example
GET	Read data	Get user details
POST	Send/Create data	Submit form
PUT	Update data	Update profile

Method	Purpose	Example
DELETE	Remove data	Delete record

---

7. AJAX using jQuery

```
$.ajax({
    url: "API_URL",
    method: "POST",
    contentType: "application/json",
    data: JSON.stringify({ key: value }),

    success: function(response) {
        console.log(response);
    },

    error: function() {
        console.log("Error");
    }
});
```

---

8. AJAX using Modern JavaScript (fetch)

```
async function loadData() {

    const res = await fetch("API_URL", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({ key: value })
    });

    const data = await res.json();
    console.log(data);
}
```

---

9. Data Format

- Most modern APIs use **JSON (JavaScript Object Notation)**

Example:

```
{
    "name": "John",
```

```
    "age": 21
}
```

---

10. Advantages of AJAX

- No page reload
- Faster interaction
- Better user experience
- Partial page updates

---

11. Limitations

- Requires JavaScript enabled
- Debugging can be complex
- Depends on API availability

API Creation (Roadmap – Beginner Friendly)

Step 1: Choose Backend Technology

Common options:

- Node.js (Express)
- .NET
- Java (Spring Boot)
- Python (FastAPI)

Step 2: Create Server

Example (Node.js):

```
const express = require("express");
const app = express();

app.use(express.json());
```

Step 3: Define Route (Endpoint)

```
app.post("/kural", (req, res) => {
    const num = req.body.num;
```

```
    res.json({
        message: "Received",
        number: num
    });
});
```

Step 4: Run Server

```
app.listen(3000, () => {
    console.log("Server running on port 3000");
});
```

Step 5: Connect Frontend (AJAX / fetch)

Frontend sends request → API responds → UI updates

Step 6: Add Database (Advanced)

- Store and retrieve real data
- Examples:
    - MySQL
    - PostgreSQL
    - MongoDB

Step 7: Deploy API

- Host backend online
- Examples:
    - Vercel
    - Render
    - AWS

AJAX vs XML vs JSON – Quick Revision Sheet

---

1. Basic Overview

Concept	Description
AJAX	Technique to communicate with server without page reload
XML	Data format used to store and transport structured data
JSON	Lightweight data format used in modern APIs

---

2. Purpose

Concept	Purpose
AJAX	Send/receive data asynchronously
XML	Store and transport structured data
JSON	Store and exchange data efficiently

---

3. Role in Web Development

Concept	Role
AJAX	Handles communication between frontend and backend
XML	Early data format used in AJAX
JSON	Modern data format used with AJAX

---

4. Syntax Comparison

XML Example

```xml
<user>
    <name>John</name>
    <age>21</age>
</user>
```

JSON Example

```json
{
    "name": "John",
    "age": 21
}
```

---

5. Key Differences (XML vs JSON)

Feature	XML	JSON
Format Type	Markup language	Data format
Syntax	Tag-based	Key-value pairs
Readability	Less readable	More readable
Size	Larger	Smaller
Parsing	Complex	Easy
Speed	Slower	Faster

---

6. AJAX Data Flow

With XML (Old)

```
User Action → AJAX Request → Server → XML Response → Parse XML → Update UI
```

With JSON (Modern)

```
User Action → AJAX Request → Server → JSON Response → Direct Use → Update UI
```

---

7. Code Comparison

AJAX with XML (Old)

```
var xhr = new XMLHttpRequest();

xhr.open("GET", "data.xml", true);

xhr.onload = function() {
    var xml = xhr.responseXML;
    var name = xml.getElementsByTagName("name")[0].childNodes[0].nodeValue;
    console.log(name);
};

xhr.send();
```

AJAX with JSON (Modern)

```
fetch("data.json")
    .then(res => res.json())
    .then(data => {
        console.log(data.name);
    });
```

---

8. Advantages Summary

Concept	Advantages
AJAX	No page reload, faster interaction
XML	Structured, extensible
JSON	Lightweight, easy to use, fast

---

9. Current Usage

Concept	Usage Today
AJAX	Widely used (core concept)
XML	Limited (legacy systems, SOAP)
JSON	Standard for modern APIs

---

10. Final Understanding

- AJAX is a **technique**
- XML and JSON are **data formats**
- Modern web apps use:
    - AJAX + JSON
- XML is mostly replaced due to complexity

---

11. One-Line Summary

> AJAX handles communication, while XML and JSON define how data is structured and transferred.